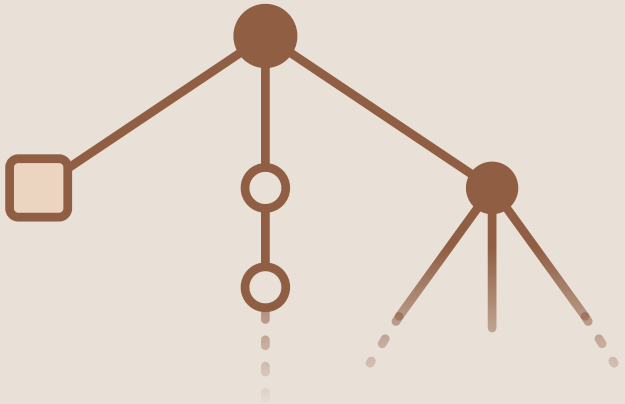


Representing Nonterminating Computation using iTrees

Philippe Dumonet

SF Lean | 2026

Real Languages are Messy

- Effects

```
let config = fs::read_to_string("config.toml")?;
fs::write("config.bak", &config)?;
```

- Nontermination

```
loop {
    let request = next_connection().await;
    respond(request).await;
}
```

- Nondeterminism and Undefined Behavior

```
let mut x = Box::new(0u32);
unsafe { *Box::as_mut_ptr(&mut x).add(1) = 2 };
```

Our Toy Language

Definition

```
inductive Expr where
  | lit (n: Nat) | var (id: Nat)
  | add (x y : Expr) | sub (x y : Expr) | lt (x y : Expr)
  | alloc (size : Expr) | deref (addr : Expr)
  | read

inductive Stmt where
  | assign (id : Nat) (v : Expr)
  | ifThenElse (cond : Expr) (zero : List Stmt) (nonZero : List Stmt)
  | while (cond : Expr) (body : List Stmt)
  | store (addr v : Expr)
  | print (v : Expr)

abbrev Program := List Stmt
```

Example Program

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```

Fueled Interpreter

```
abbrev Fuel := Nat

inductive Error where | ub | outOfFuel | invalidVar

def eval (o : Oracle) (σ : State) : Expr → Except Error (Nat × State)

def exec (o : Oracle) (σ : State) : Fuel → Program → Trace × Option Error
```

- Nontermination implicit (no fuel value that doesn't error)
- Nondeterminism and effect results require fixed implementations upfront

Fueled Interpreter

```
abbrev Fuel := Nat

inductive Error where | ub | outOfFuel | invalidVar

def eval (o : Oracle) (σ : State) : Expr → Except Error (Nat × State)

def exec (o : Oracle) (σ : State) : Fuel → Program → Trace × Option Error
```

- Nontermination implicit (no fuel value that doesn't error)
- Nondeterminism and effect results require fixed implementations upfront

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```

oracle: read ↦ 3

fuel

6

Relational Big-Step

```
inductive BigStep : Stmt → State → State → Trace → Prop   -- ⟨s, σ⟩ ⇓ ⟨σ', t⟩
```

$$\frac{\langle s_1, \sigma\rangle \Downarrow \langle \sigma_1, t_1\rangle \quad \langle s_2, \sigma_1\rangle \Downarrow \langle \sigma_2, t_2\rangle}{\langle s_1 \,;\, s_2, \sigma\rangle \Downarrow \langle \sigma_2, t_1 \mathbin{++} t_2\rangle}\ (\textbf{seq}) \qquad \frac{a \in \mathbb{N} \quad \langle e, \sigma\rangle \Downarrow n \quad \forall\, i < n,\ \sigma.\mathrm{heap}\ (a + i) = \mathrm{none}}{\langle x := \textbf{alloc}\ e, \sigma\rangle \Downarrow \langle \sigma[a..a+n \mapsto_h 0][x \mapsto a], []\rangle}\ (\textbf{alloc})$$

$$\frac{v \in \mathbb{N}}{\langle x := \textbf{read}, \sigma\rangle \Downarrow \langle \sigma[x \mapsto v], [\textbf{in}\ v]\rangle}\ (\textbf{read})$$

- Nondeterminism & effects are "free" (e.g. `a` in `alloc`, `v` in `read`)
- Nontermination handled suboptimally
  - In Naive Big-Step: Stuck State (no derivation)
  - Fix 1: Add explicit nontermination judgments (duplicating rules)
  - Fix 2: Small-step-like relation

Small-Step

```
abbrev Cfg := Stmt × State                          -- the program is part of the config
inductive Step  : Cfg → Option Event → Cfg → Prop    -- none = silent τ step
inductive Steps : Cfg → Trace → Cfg → Prop           -- collect the labels
```

```
⟨while c s, σ⟩  −τ→      ⟨if c then (s ; while c s) else skip, σ⟩
⟨print e, σ⟩    −out v→  ⟨skip, σ⟩                    if ⟦e⟧σ = v
⟨s₁ ; s₂, σ⟩    −ℓ→      ⟨s₁' ; s₂, σ'⟩               if ⟨s₁, σ⟩ −ℓ→ ⟨s₁', σ'⟩
```

- Direct reasoning about nontermination
- Nondeterminism & effects are still "free"
- Requires explicitly modeling state transitions
- Resulting execution not immediately clear (⇒ bug-prone)
    - Lean 4 auto-implicits can obscure what is being quantified over
    - Accidental nondeterminism (several valid derivations for one state)
    - Unintended UB (via stuck states)

Big Picture Comparison

	Fueled Interpreter	Big-Step Relation	Small-Step Relation
Executable?	✅	❌	❌
Nontermination	👎 Awkward quantification over `fuel`	👎 Stuck State / UB / Duplicate Coinductive Rules	✅ Infinite Step Sequence
Effects & Nondeterminism	👎 Preset Responses / Concrete Model	✅ Simple Derivation Pre-Condition	✅ Simple Derivation Pre-Condition
Undefined Behavior (UB)	✅ Explicit Error Kind	👎 Stuck State	👎 Stuck State

Big Picture Comparison

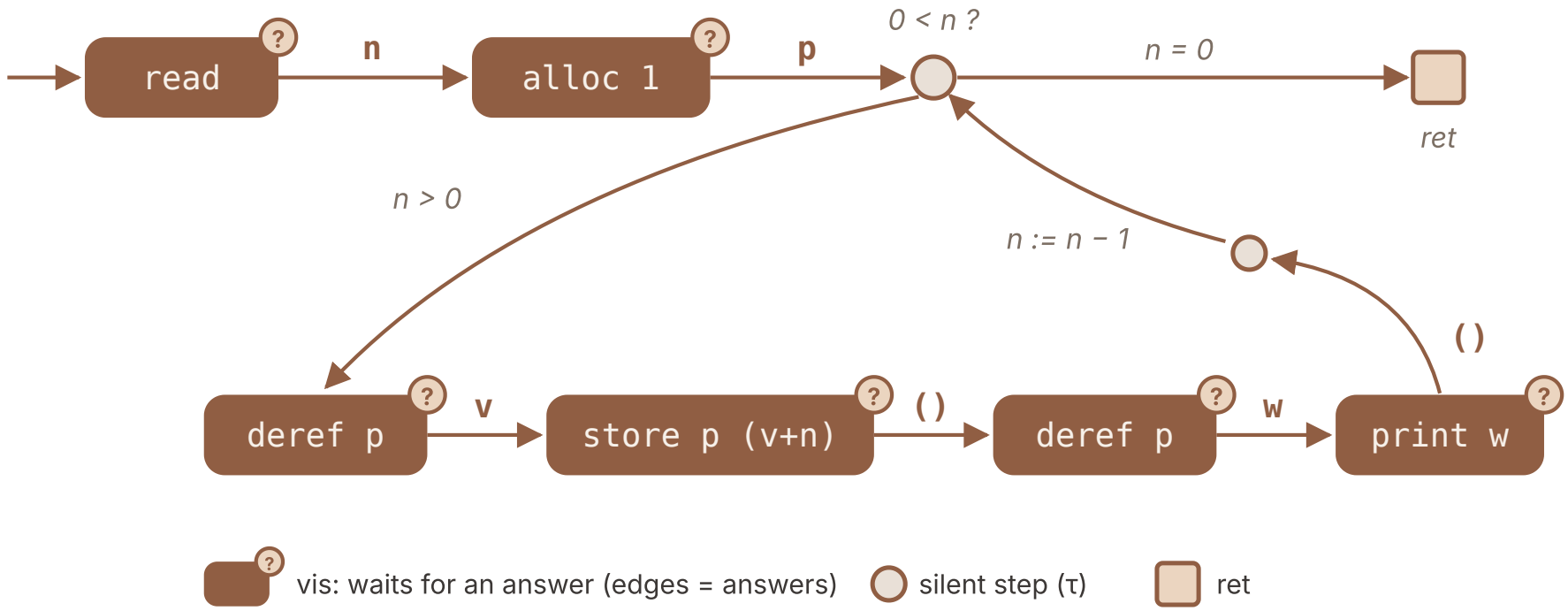
	Fueled Interpreter	Big-Step Relation	Small-Step Relation
Executable?	✅	❌	❌
Nontermination	👎 Awkward quantification over `fuel`	👎 Stuck State / UB / Duplicate Coinductive Rules	✅ Infinite Step Sequence
Effects & Nondeterminism	👎 Preset Responses / Concrete Model	✅ Simple Derivation Pre-Condition	✅ Simple Derivation Pre-Condition
Undefined Behavior (UB)	✅ Explicit Error Kind	👎 Stuck State	👎 Stuck State

What does our Ideal Version look like?

- Executable
- Trivially Nonterminating
- Abstract / Modular Effects
- UB Explicit

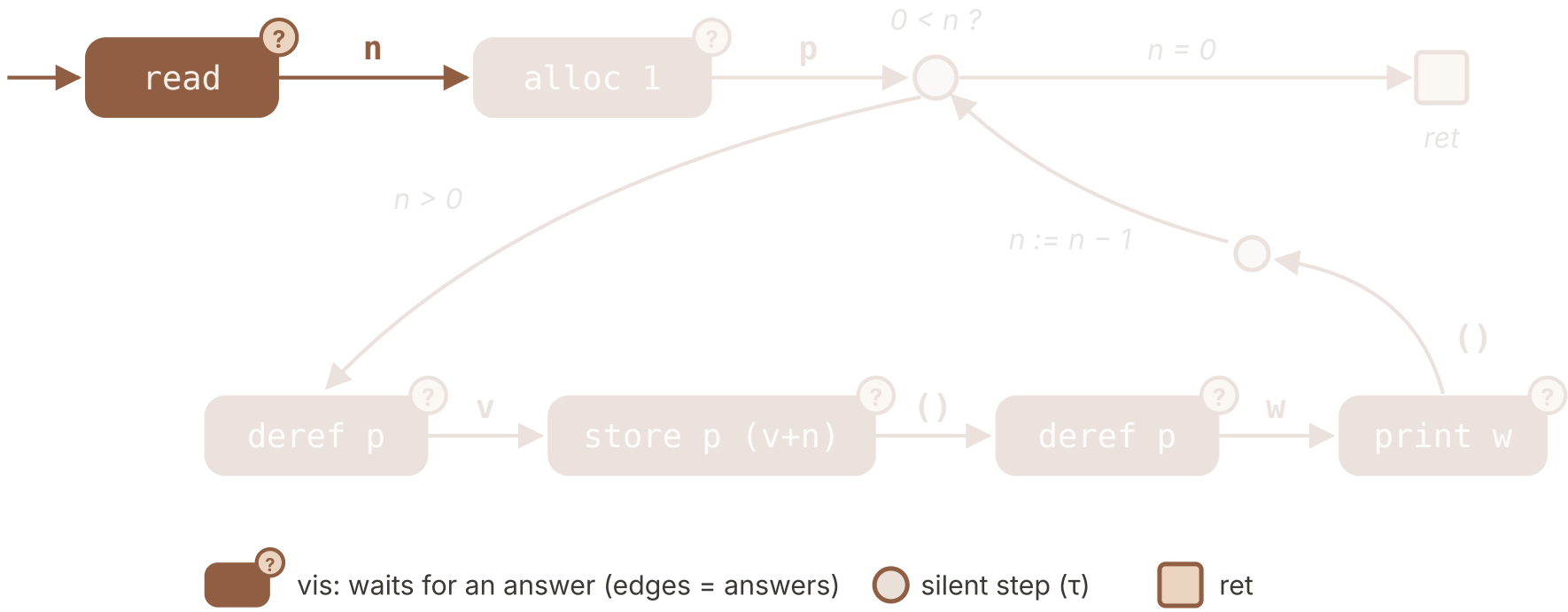
iTrees (Interaction Trees) I

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



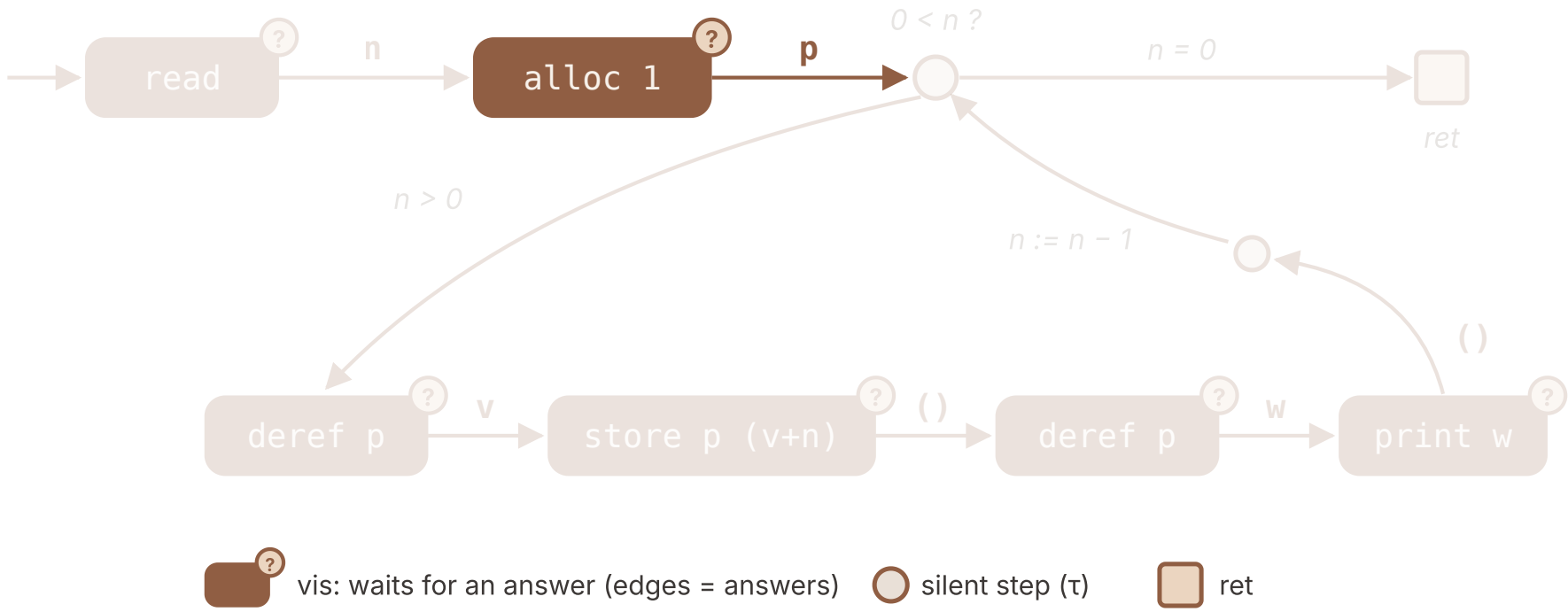
iTrees (Interaction Trees) I

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



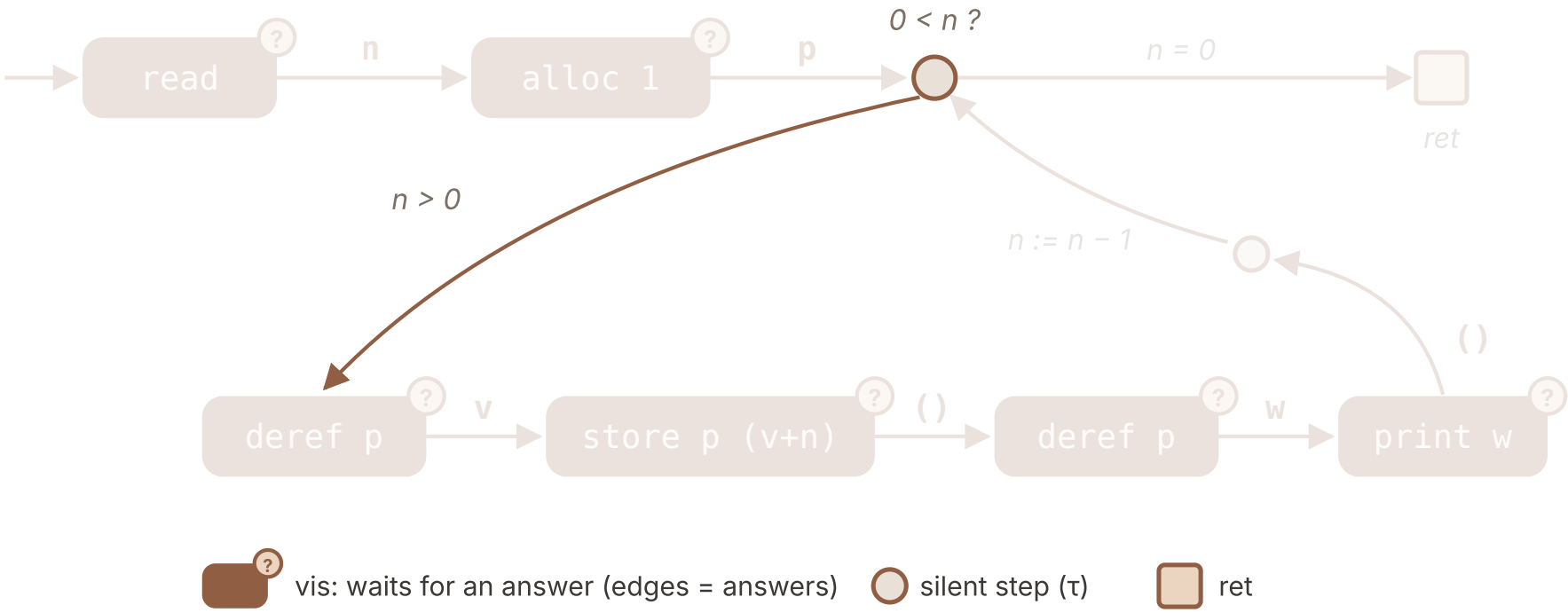
iTrees (Interaction Trees) I

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



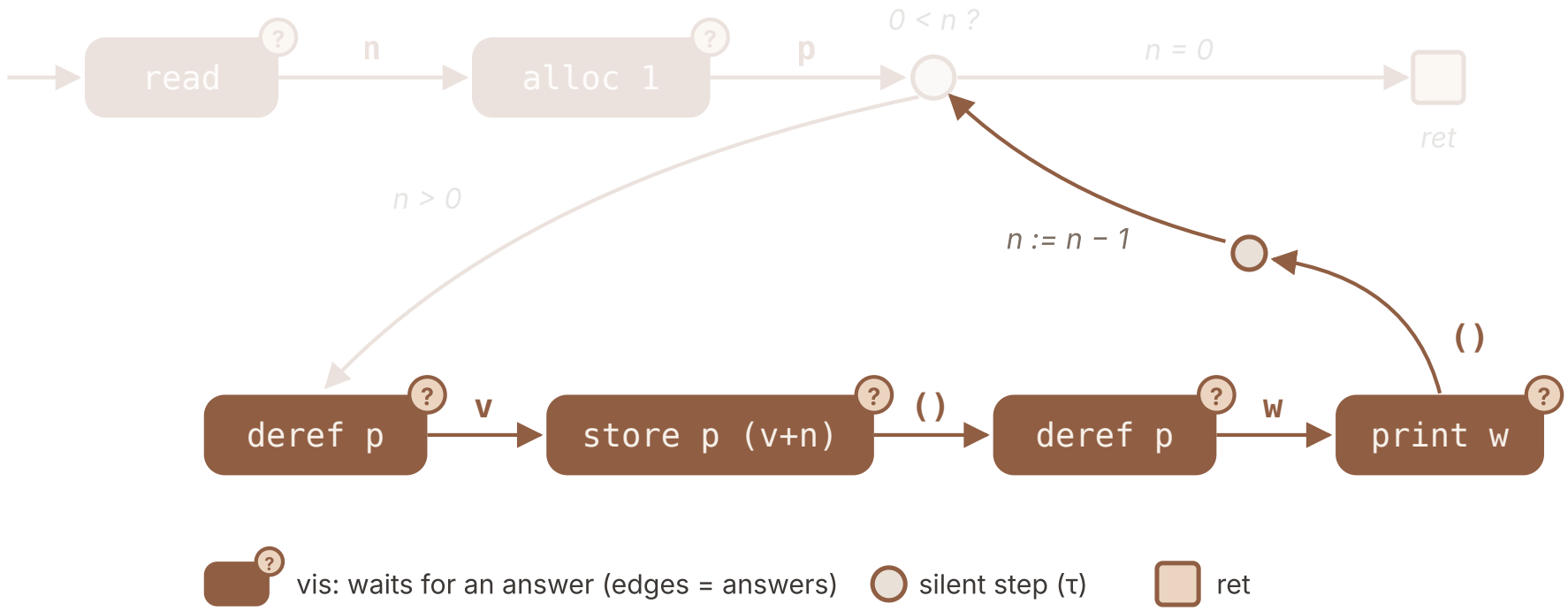
iTrees (Interaction Trees) I

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



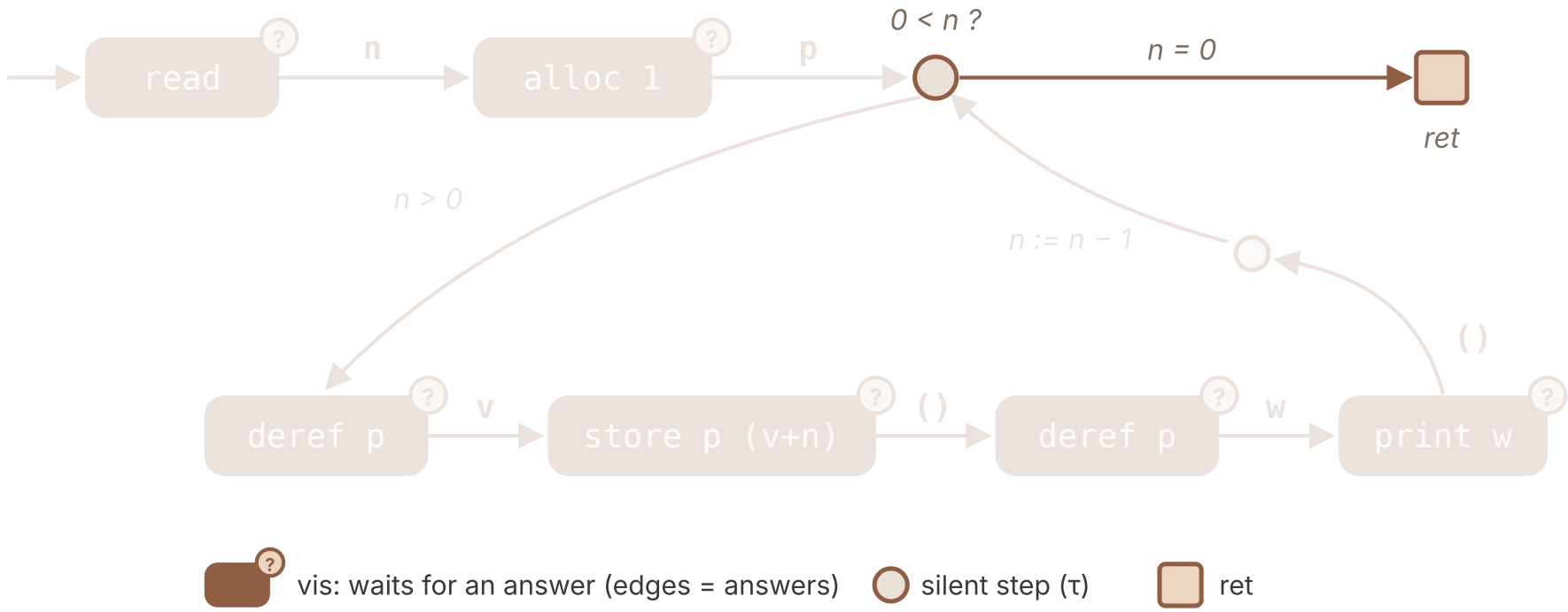
iTrees (Interaction Trees) I

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



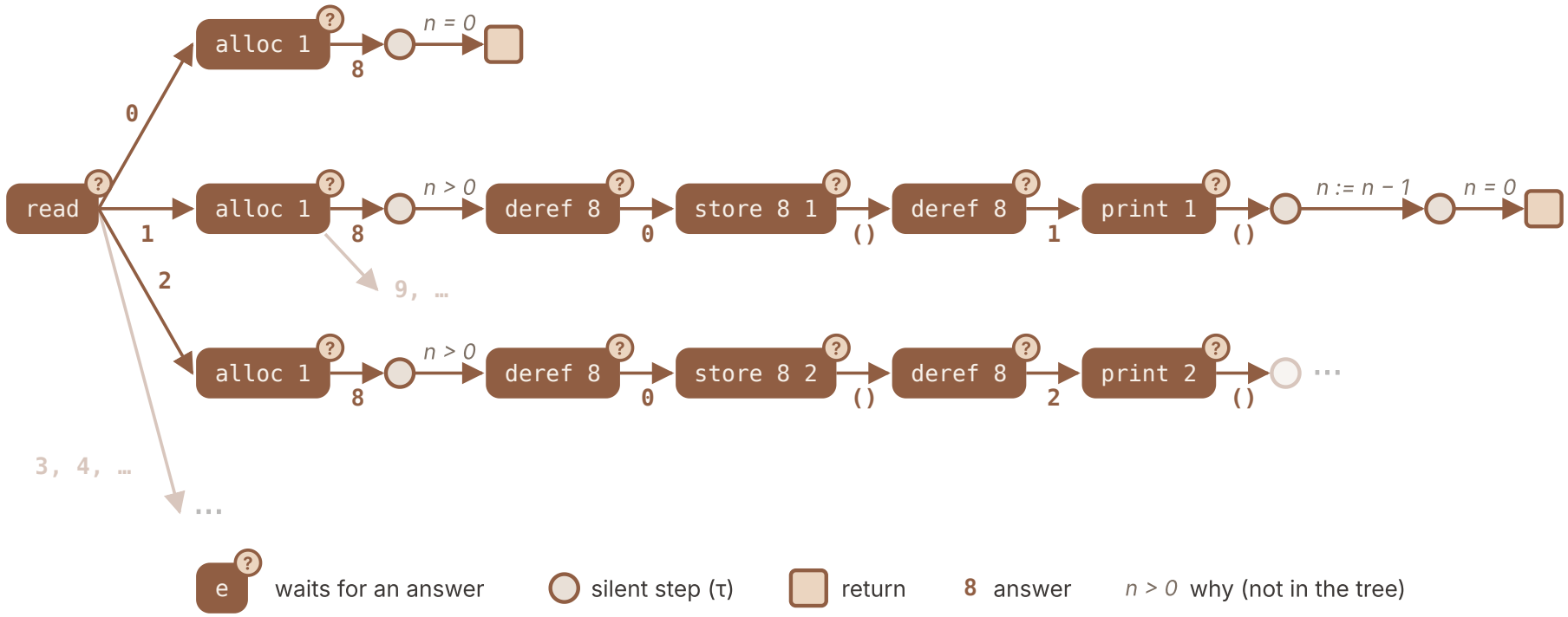
iTrees (Interaction Trees) I

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



iTrees II

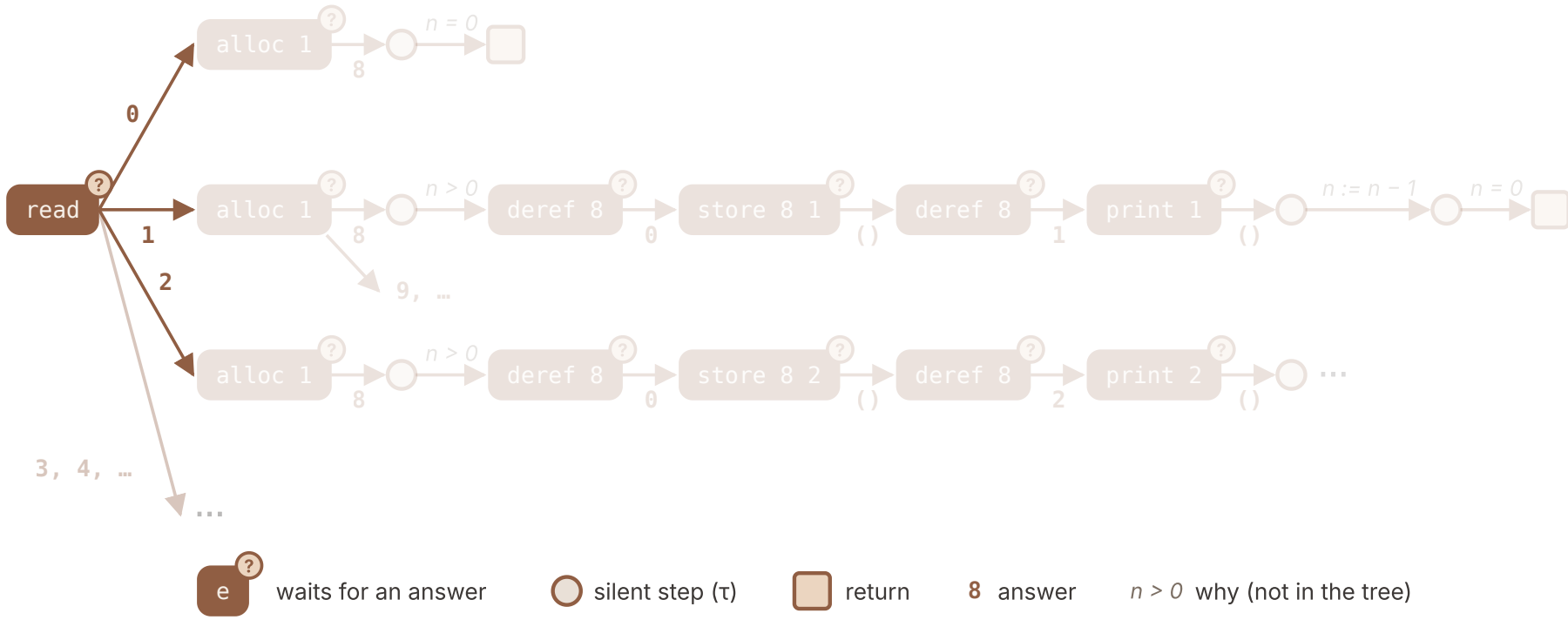
```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



Every effect is root branching on the response, unfolding the tree on the result

iTrees II

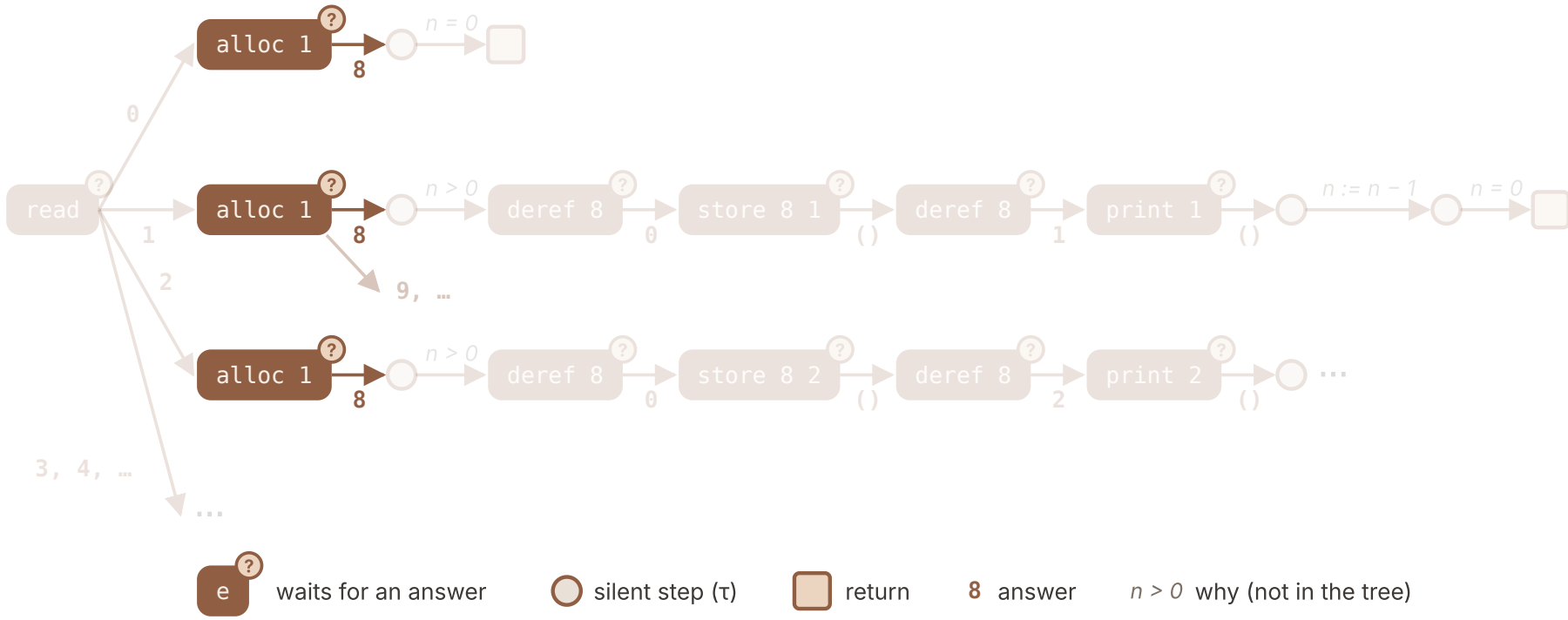
```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



Every effect is root branching on the response, unfolding the tree on the result

iTrees II

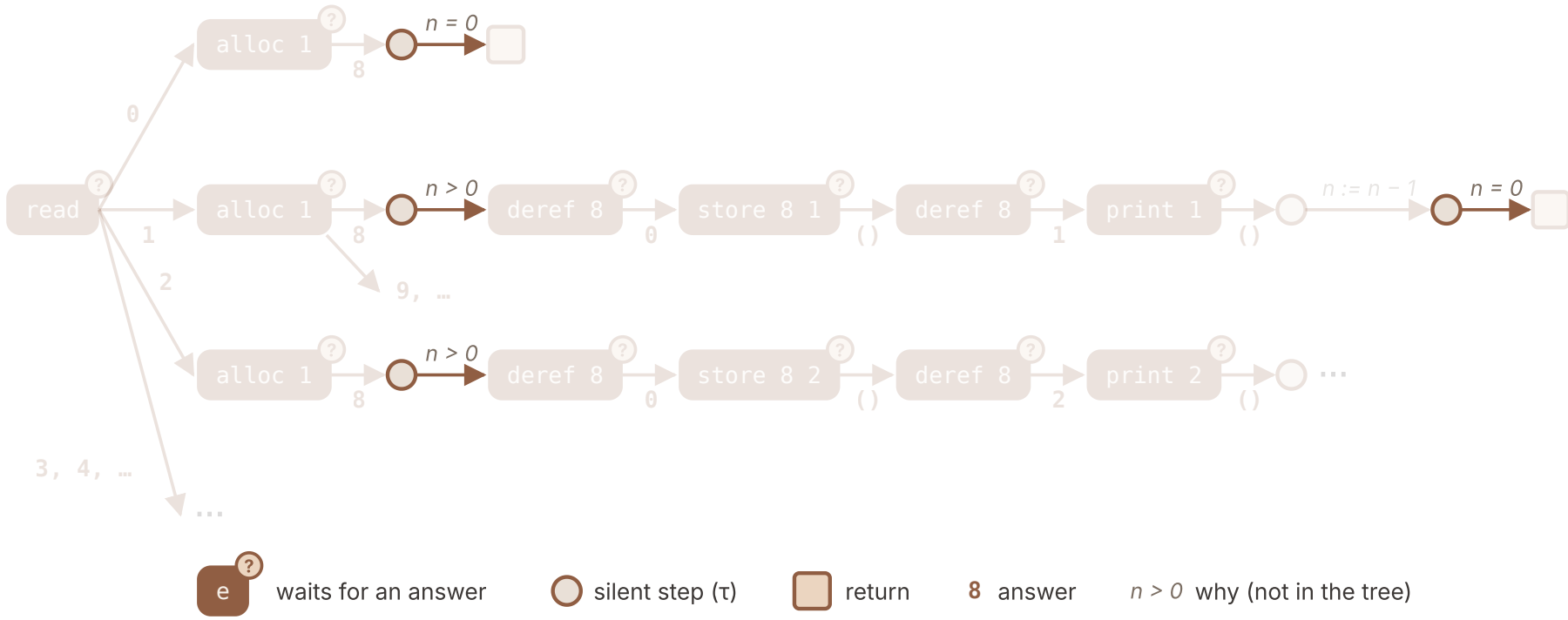
```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



Every effect is root branching on the response, unfolding the tree on the result

iTrees II

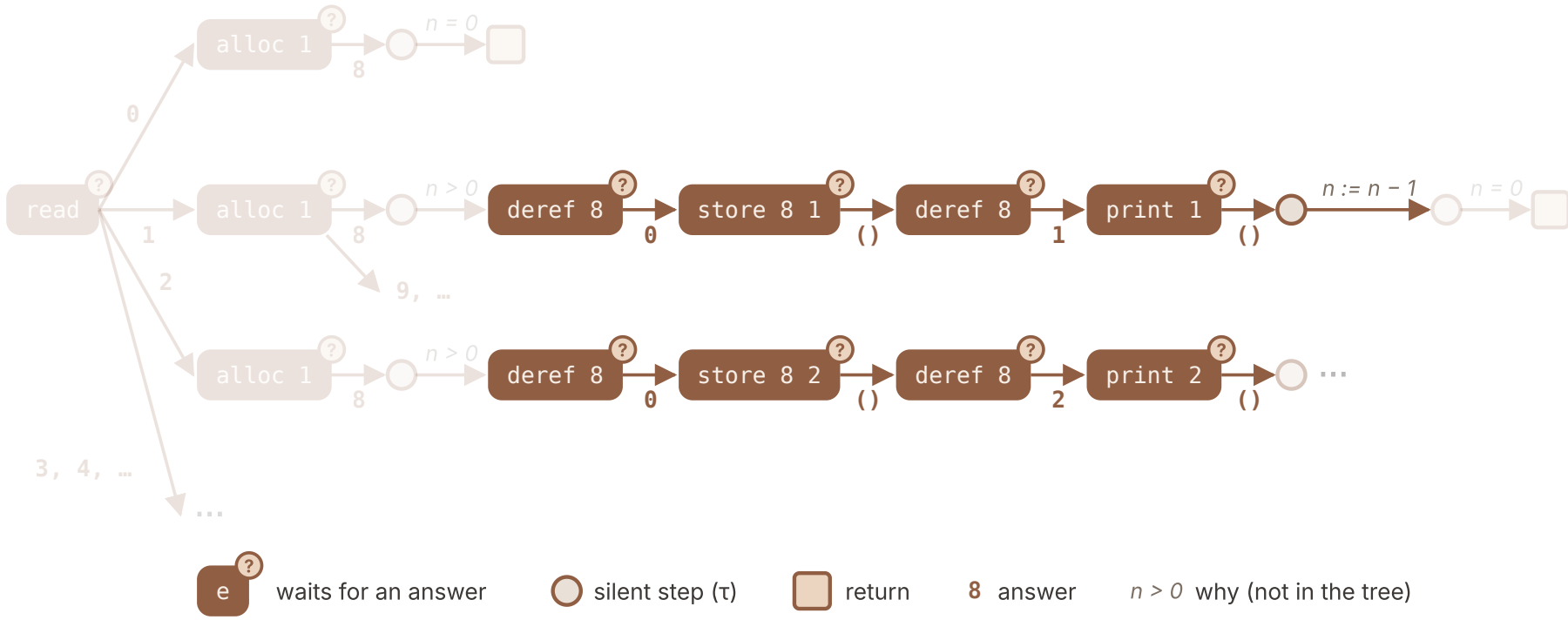
```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



Every effect is root branching on the response, unfolding the tree on the result

iTrees II

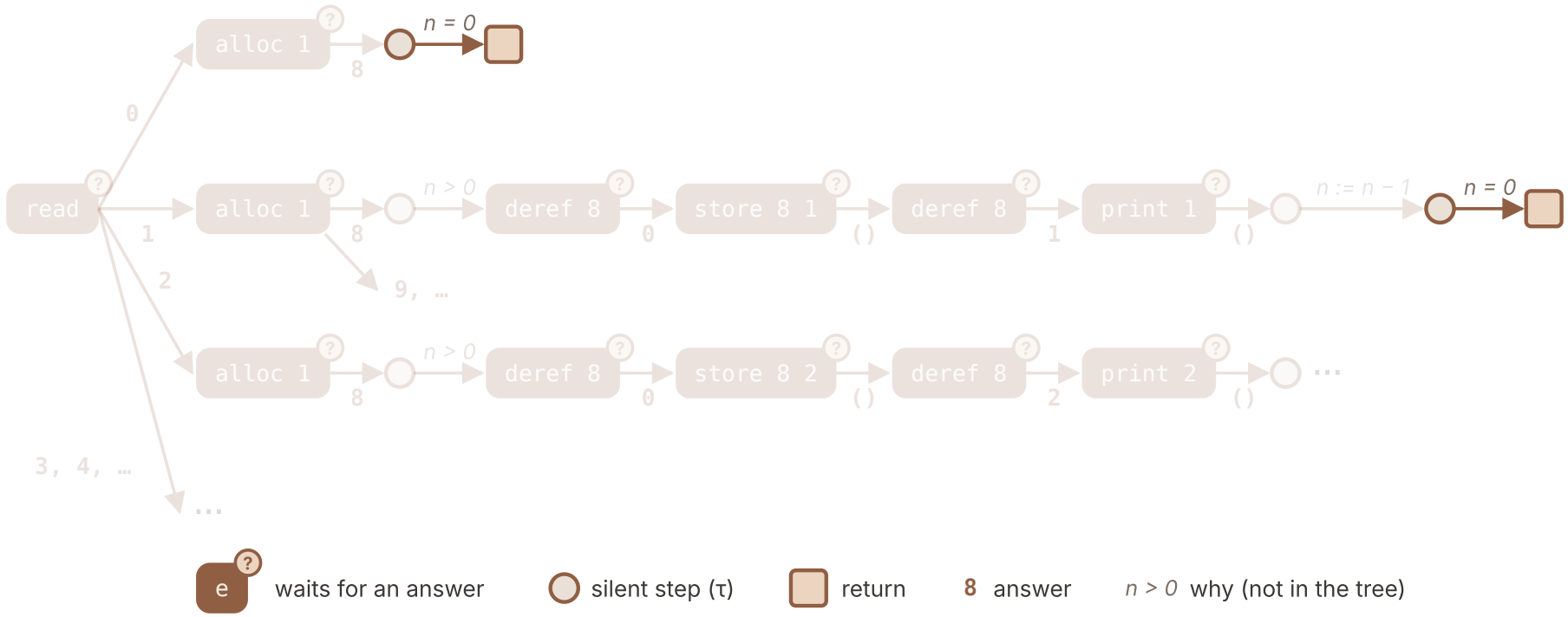
```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



Every effect is root branching on the response, unfolding the tree on the result

iTrees II

```
n := read;
p := alloc 1;
while 0 < n do
  store p (deref p + n);
  print (deref p);
  n := n - 1
```



Every effect is root branching on the response, unfolding the tree on the result

iTree Based Interpreter

```
def eval (vars : Vars) : Expr → ITree Ev Nat
  | .lit n     => pure n
  | .var x     => pure (vars x)
  | .add a b   => do return (← eval vars a) + (← eval vars b)
  | .read      => read
  | .alloc e   => do alloc (← eval vars e)
  | .deref e   => do deref (← eval vars e)
  -- | .sub a b => ... | .lt a b => ...

def exec : Vars × Stmt → ITree Ev Vars := ITree.fix fun exec (vars, s) => match s with
  | .assign x e  => do return vars.set x (← eval vars e)
  | .print e     => do print (← eval vars e); return vars
  | .while c ss  => do
      let mut vars := vars
      while (← eval vars c) ≠ 0 do
        for s in ss do
          vars ← exec (vars, s)
      return vars
  | .store a e   => do write (← eval vars a) (← eval vars e); return vars
  | .ifThenElse c z nz => do
      let branch := if (← eval vars c) = 0 then z else nz
      let mut vars := vars
      for s in branch do
        vars ← exec (vars, s)
      return vars
```

What iTrees Give You

- Nontermination (Unbounded Loops & Recursion)
- Quasi-Executable with Modular Effects (executable shape fixed, only effects need concrete definitions)
- Explicit UB (Explicit Kind / Non-resumable Effect)

How iTrees Work

```
/---- iTrees are just fancy State Machines ----/

inductive Step (Effect : Type → Type) (Res State : Type) : Type 1 where
  | ret (r : Res)
  | tau (next : State)
  | vis {A : Type} (e : Effect A) (k : A → State)

structure ITree (E : Type → Type) (R : Type) : Type 1 where
  σ : Type
  now : σ
  step : σ → Step E R σ

/---- iTrees as Monads ----/

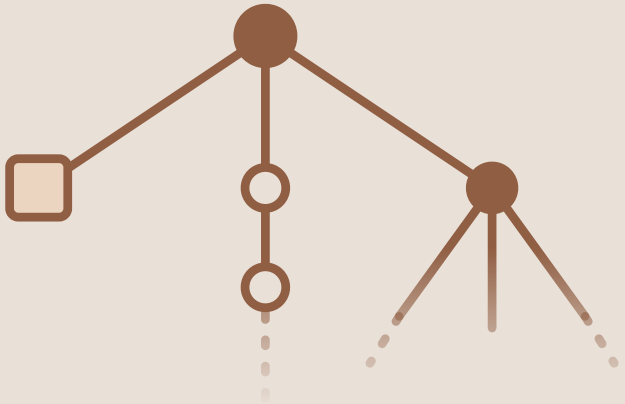
def ITree.pure (r : R) : ITree E R := ⟨Unit, (), fun () => .ret r⟩

def ITree.bind (t : ITree E A) (g : A → ITree E B) : ITree E B where
  σ := t.σ ⊕ (Σ a, (g a).σ)
  now := .inl t.now
  step
    | .inl s => match t.step s with
      | .ret a => .tau (.inr ⟨a, (g a).now⟩)
      | .tau s' => .tau (.inl s')
      | .vis e k => .vis e (fun x => .inl (k x))
    | .inr ⟨a, s⟩ => match (g a).step s with
      | .ret b => .ret b
      | .tau s' => .tau (.inr ⟨a, s'⟩)
      | .vis e k => .vis e (fun x => .inr ⟨a, k x⟩)
```

Recap

It's all about the Specification

- Bugs in the Specification undermine the Soundness of your Verification
- Typical language formalization approaches are suboptimal
- With AI **you are the bottleneck** ⇒ anything that makes critical code less painful to audit is a win
- iTrees are great



Thanks for listening!

philippe@dumo.net

@real_philogy

slides: github.com/Philogy/2026-sflean-itree-presentation-slides

Appendix: Sources

- **Interaction Trees (Xia et al., POPL 2020)**
  Coinductive free monad over events; eutt, interp, Coq library
- **General Recursion via Coinductive Types (Capretta, LMCS 2005)**
  The delay monad: iTrees without events
- **Choice Trees (Chappe et al., POPL 2023)**
  iTrees plus nondeterministic branching; CCS and cooperative concurrency
- **reilabs/freigen**
  Reilabs' Lean 4 iTree and effects library
- **leodemoura/ETAPSTutorial2026**
  Slide look adapted from Leo de Moura's tutorial
- **leanprover/verso-slides**
  Verso genre these slides are built with